Task 1: Create a GitHub Account

Outcome

A GitHub account was created successfully, and a public repository named devopsproject-01 was created under the account naveenlingampelli456-cmyk.

Repository URL:

https://github.com/naveenlingampelli456-cmyk/devopsproject-01

Objective

To create a GitHub account for storing source code, tracking changes, collaborating with developers, and managing DevOps practice projects using Git version control and GitHub repositories.

Prerequisites

- Internet connectivity.
- A valid email address.
- Access to a web browser.
- Git installed on the local machine.
- Basic understanding of Git and GitHub.

Step-by-Step Implementation

Step 1: Open the GitHub Website

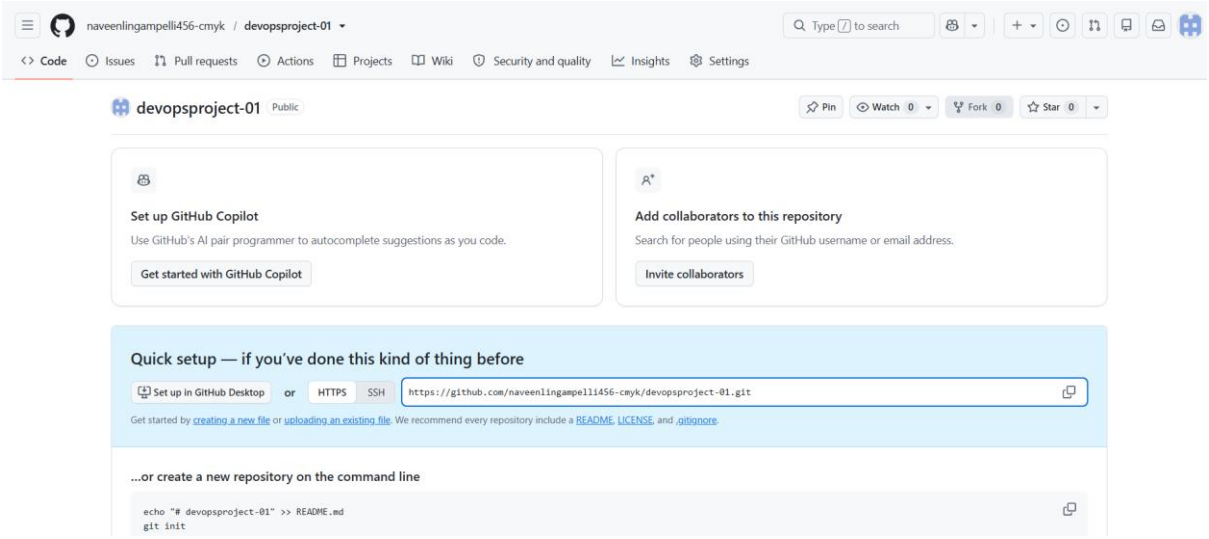
1. Open a web browser.
2. Navigate to https://github.com/.
3. Select **Sign up**.
4. Enter the required registration details.
5. Verify the email address and complete the registration process.

Step 2: Verify the GitHub Account

1. Sign in to GitHub.
2. Open the profile menu.
3. Verify that the account is accessible.
4. Confirm the account username.

Account username: naveenlingampelli456-cmyk

Screenshot Evidence



Step 3: Create a GitHub Repository

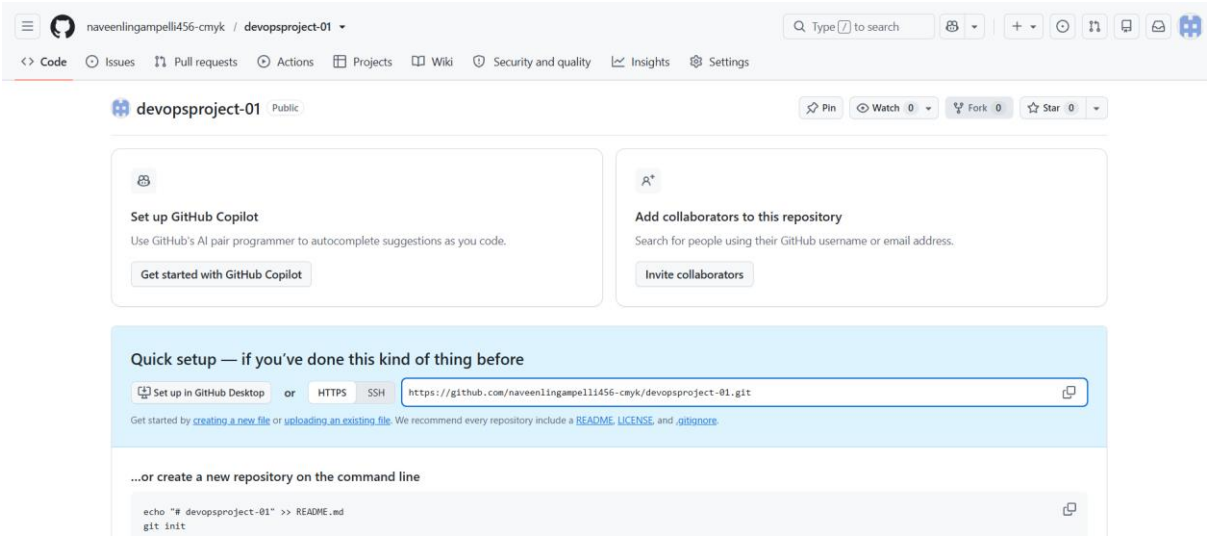
1. Sign in to GitHub.
2. Select the **New repository** option.
3. Enter the repository name.
4. Choose the repository visibility.
5. Create the repository.

Configuration details:

Parameter	**Value**
Repository name	devopsproject-01
Visibility	Public
Repository URL	https://github.com/naveenlingampelli456-cmyk/devopsproject-01
Branch used	Master

The repository was created and displayed the GitHub Quick setup instructions.

Screenshot Evidence



Validation Steps

1. Open the repository URL in a browser.
2. Verify that the repository is accessible.
3. Confirm that the repository name and account username are correct.

Expected result:

The GitHub account and repository are available, and the repository can be accessed through its URL.

Explanation

GitHub is a cloud-based platform used to host Git repositories. It allows developers to maintain source code, track changes, collaborate on projects, and manage different versions of an application. A repository contains project files and their version history.

Creating a GitHub account is an important first step in DevOps because source code and configuration files are frequently maintained in remote repositories. GitHub also supports collaboration, pull requests, issue tracking, and automation through GitHub Actions.

Real-Time Use Case

DevOps engineers use GitHub to store application code, shell scripts, Dockerfiles, Terraform configurations, Kubernetes manifests, and CI/CD pipeline definitions. Teams can collaborate on changes while maintaining a history of modifications.

Common Errors and Troubleshooting

Error	Cause	Resolution
Unable to sign in	Incorrect credentials or an unverified account	Verify the account details and complete any required verification.
Repository name unavailable	Repository name already exists in the relevant namespace	Choose another repository name.

Repository not accessible	Incorrect URL or insufficient permissions	Verify the repository URL and account permissions.
Repository created with the wrong visibility	Incorrect visibility selection	Review repository settings and change visibility if appropriate.

Conclusion

A GitHub account was created and a public repository named devopsproject-01 was established. The repository provides a remote location for storing and managing files used in the DevOps practical exercises.

Task 2: Enable Multi-Factor Authentication (MFA) on GitHub Using a Mobile App

Outcome

Two-factor authentication (2FA) was successfully enabled on the GitHub account using an authenticator app on a mobile device.

The confirmation screen displayed the message:

"Two-factor authentication (2FA) is now enabled for your GitHub account."

Objective

To improve GitHub account security by requiring a second authentication factor in addition to the password. MFA helps protect repositories and account settings against unauthorized access when a password is compromised.

Prerequisites

- An active GitHub account.
- Access to the registered email address.
- A mobile device.
- An authenticator application capable of generating verification codes.
- Internet connectivity.
- Access to GitHub account security settings.

Step-by-Step Implementation

Step 1: Sign In to GitHub

1. Open https://github.com/.
2. Sign in using the registered account.
3. Open the profile menu.
4. Navigate to **Settings**.

Step 2: Open Password and Authentication Settings

Navigate through the following path:

GitHub → Profile Menu → Settings → Password and authentication

Locate the section for two-factor authentication.

Step 3: Configure Authenticator-App MFA

1. Start the two-factor authentication setup.
2. Select the authenticator application option.
3. Open the authenticator app on the mobile device.
4. Scan the QR code displayed by GitHub, or manually enter the setup key if that option is provided.
5. Enter the verification code generated by the authenticator app.
6. Complete the verification process.

The authenticator app generates time-based verification codes that change periodically.

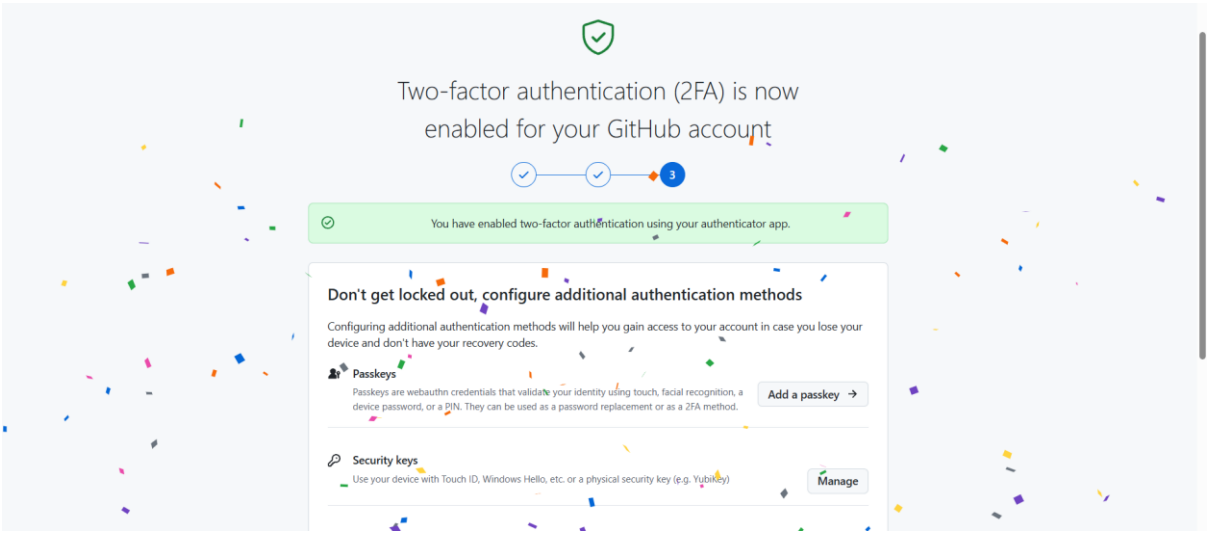
Step 4: Complete MFA Setup

After the verification succeeds, GitHub confirms that two-factor authentication is enabled.

The confirmation screenshot supplied for this assignment shows that:

- Two-factor authentication is enabled.
- The authenticator-app method was used.
- GitHub recommends configuring additional authentication methods.

Screenshot Evidence



Step 5: Secure Account Recovery

1. Save the GitHub recovery codes in a secure location if provided.
2. Keep the authenticator app accessible.

3. Consider configuring an additional supported authentication method.
4. Never share recovery codes, authenticator setup secrets, or verification codes.

Validation Steps

1. Open GitHub account settings.
2. Navigate to **Password and authentication**.
3. Verify that two-factor authentication is enabled.
4. Confirm that the authenticator-app method is configured.
5. If appropriate, sign in again to verify that the second authentication factor is requested.

Expected result:

GitHub confirms that 2FA is enabled using an authenticator application.

Explanation

Multi-factor authentication adds a second verification step to the account sign-in process. In this exercise, the password is combined with a time-based verification code generated by a mobile authenticator application. This reduces the risk of unauthorized access if the password is exposed.

For DevOps engineers, GitHub security is especially important because repositories may contain application code, deployment configurations, automation scripts, and infrastructure definitions. Strong authentication helps protect these assets and prevents unauthorized changes to source code and CI/CD workflows.

Real-Time Use Case

Organizations use MFA to secure GitHub accounts belonging to developers, administrators, and DevOps engineers. It provides an additional security layer when users access repositories, change security settings, manage access permissions, or contribute to deployment pipelines.

Common Errors and Troubleshooting

Error	Cause	Resolution
Invalid authentication code	Expired code or incorrect device time	Wait for a new code and ensure the device time is synchronized.
QR code cannot be scanned	Camera issue or incomplete QR display	Retry scanning or use the manual setup option if available.
Lost mobile device	Authenticator app is unavailable	Use a saved recovery code or another configured recovery method.
MFA setup does not complete	Verification was not successful	Restart the setup and carefully follow the verification steps.
Locked out of account	No access to the authenticator or recovery method	Follow GitHub's official account recovery process.

Conclusion

Two-factor authentication was successfully enabled on the GitHub account using a mobile authenticator app. The confirmation screenshot provides evidence of successful configuration. Recovery information should be stored securely to prevent account lockout.

Task 3: Execute Git Commands Demonstrated in the Video

Outcome

Git commands were executed in Git Bash on a Windows machine to initialize a local repository, create files, stage changes, commit files, inspect repository status and history, delete a tracked file, configure the remote repository, synchronize remote changes, and push local commits to GitHub.

The local repository was connected to the remote GitHub repository devopsproject-01, and successful push operations were recorded in the screenshots.

Objective

To gain hands-on experience with Git version control and understand how local repositories interact with remote GitHub repositories. This exercise covers the basic Git workflow: working directory, staging area, commits, branches, and remote synchronization.

Prerequisites

- Git installed on Windows.
- Git Bash available.
- A GitHub account.
- A GitHub repository named devopsproject-01.
- SSH key configured on the local machine and registered with GitHub.
- Access to the local working directory D:\devops2\git-practice.

Step-by-Step Implementation

Step 1: Open Git Bash and Navigate to the Working Directory

The working directory used in the exercise was:

cd /d/devops2/git-practice

Check the current location:

pwd

Expected location:

/d/devops2/git-practice

Initially, the git status command returned:

fatal: not a git repository (or any of the parent directories): .git

This occurred because the directory had not yet been initialized as a Git repository.

Step 2: Initialize the Local Git Repository

Run:

git init

The command initializes a new Git repository in the current directory.

The observed output was similar to:

Initialized empty Git repository in D:/devops2/git-practice/.git/

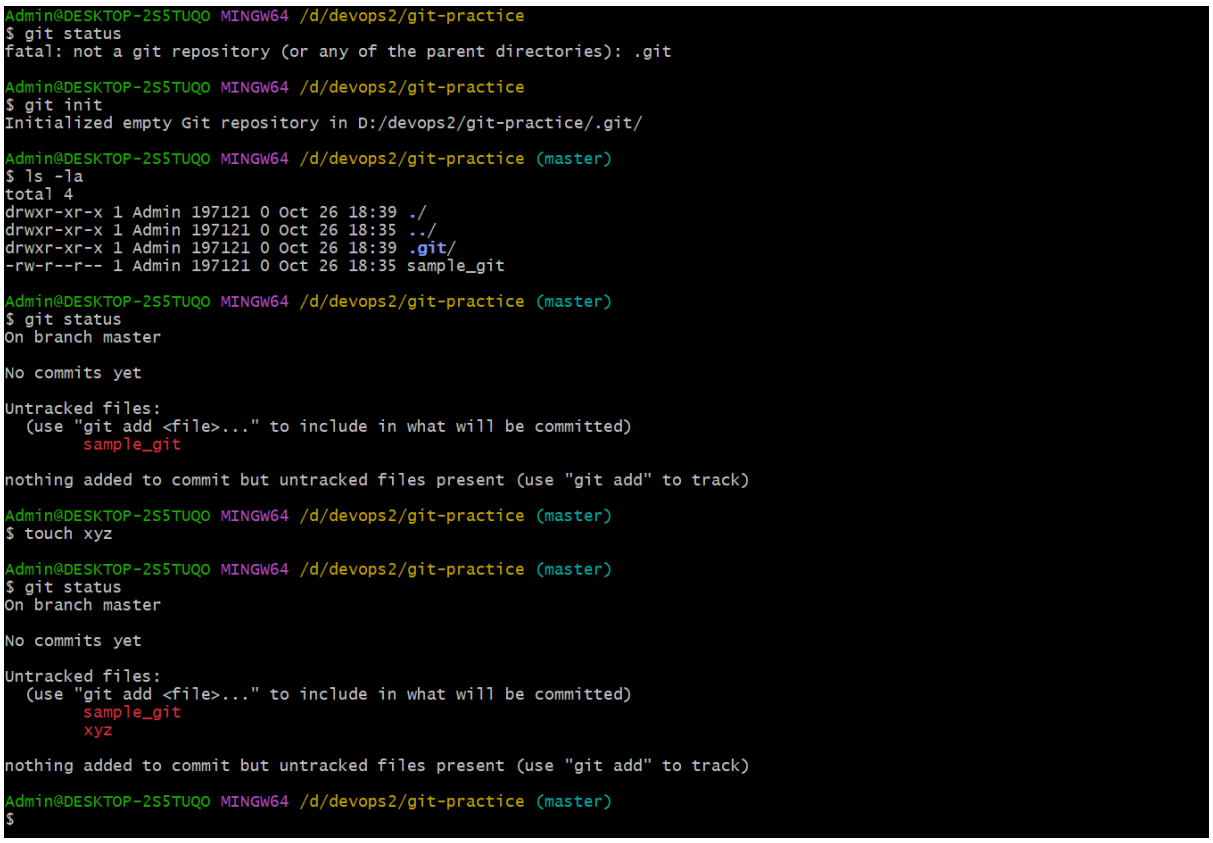
Verify the repository status:

git status

The output showed that the repository was on the master branch and that no commits existed yet.

Screenshot Evidence

Screenshot 2:



Step 3: Inspect Files in the Working Directory

List files, including hidden entries:

ls -la

The directory initially contained the file sample_git. Additional files were created during the exercise.

Create a file named xyz:

touch xyz

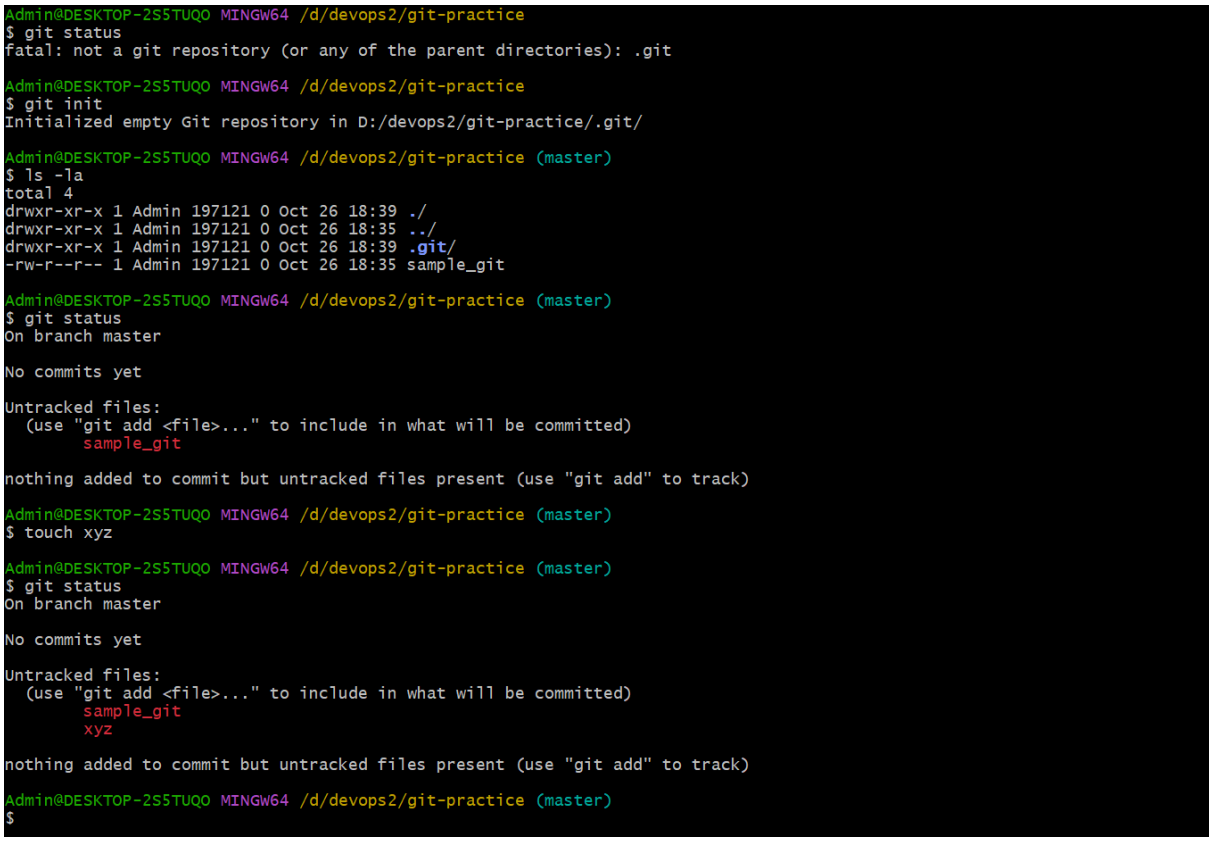
Check the status:

git status

Git displayed sample_git and xyz as untracked files.

Screenshot Evidence

Screenshot 3:



Step 4: Inspect Git Configuration

Display Git configuration values:

git config --list

The screenshot showed the following contributor configuration:

user.name=Naveen Kumar

The configured email was visible in the original terminal screenshot. For privacy, it is omitted from this document.

These values identify the author of future commits unless overridden by repository-specific configuration.

To check the configured author name and email individually, use:

git config user.name

git config user.email

Screenshot Evidence

Screenshot 4:

```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Naveen Kumar
user.email=naveen.lingampelli21@gmail.com
```

Step 5: Stage Selected Files

Stage the initial files:

git add sample_git xyz

Verify the staged changes:

git status

The output showed sample_git and xyz under **Changes to be committed**.

Next, create additional files:

touch a b c d e f

Stage all files in the current directory:

git add .

Check the repository status:

git status

The output showed the new files staged for commit.

Screenshot Evidence

Screenshot 5:

```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git add sample_git xyz

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   sample_git
        new file:   xyz
```

```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ touch a b c d e f

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   sample_git
        new file:   xyz

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        a
        b
        c
        d
        e
        f


Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git add .

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   a
        new file:   b
        new file:   c
        new file:   d
        new file:   e
        new file:   f
        new file:   sample_git
        new file:   xyz
```

Step 6: Create the First Commit

Run:

git commit -m "new files added"

The screenshot showed a successful commit similar to:

[master (root-commit) 5989aa5] new files added

8 files changed, 0 insertions(+), 0 deletions(-)

The commit recorded the eight files tracked at that point.

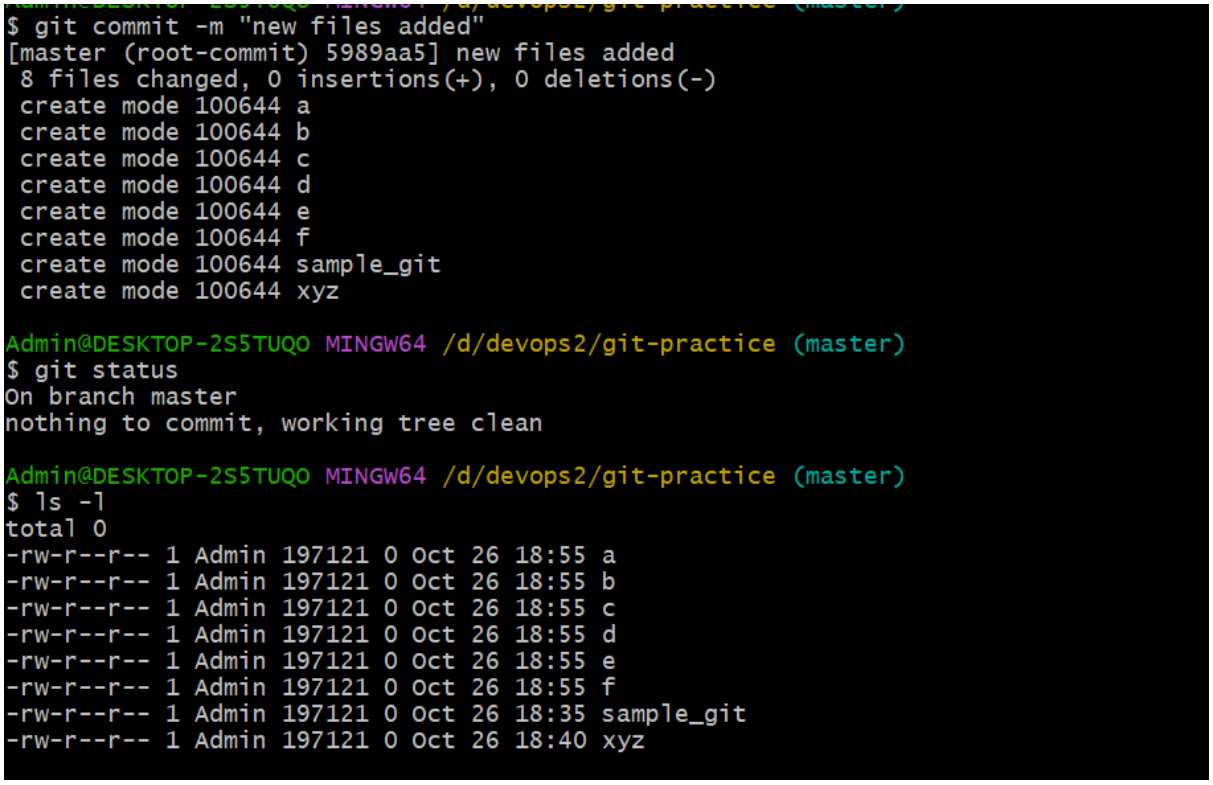
Verify the repository status:

git status

Expected result:

nothing to commit, working tree clean

Screenshot Evidence



Step 7: Create and Commit Another File

Create a file named z:

touch z

Stage and commit it:

git add .

git commit -m "new file added"

Verify the repository status:

git status

Inspect the commit history:

git log

The screenshot showed the commits and their author details.

Screenshot Evidence

```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ touch z

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git add . ; git commit -m "new file added"
[master b08eab8] new file added
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 z

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git status
On branch master
nothing to commit, working tree clean

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ ls -l
total 0
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 a
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 b
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 c
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 d
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 e
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 f
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:35 sample_git
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:40 xyz
-rw-r--r-- 1 Admin 197121 0 Oct 26 19:04 z

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git log
commit b08eab8b01924343ba1ea3ba53daa7d4d6588018 (HEAD -> master)
Author: naveenkumar <naveen.lingampelli21@gmail.com>
Date:   Mon Oct 26 19:04:47 2026 +0530

    new file added

commit 5989aa53c97b197c5482ddc72be5db595074291a
Author: naveenkumar <naveen.lingampelli21@gmail.com>
Date:   Mon Oct 26 18:59:58 2026 +0530

    new files added
```

Step 8: Delete a File and Commit the Deletion

Delete the file xyz:

rm -rf xyz

For a single regular file, rm xyz would also be sufficient. The -r option is unnecessary for a file.

Stage the deletion and create a commit:

git add .

git commit -m "deleted"

Push the changes to the remote repository:

git push origin master

The screenshot showed the deletion commit being pushed to GitHub successfully.

Screenshot Evidence

```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ rm -rf xyz

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ ls -la
total 12
drwxr-xr-x 1 Admin 197121 0 Oct 26 19:53 ./
drwxr-xr-x 1 Admin 197121 0 Oct 26 18:35 ../
drwxr-xr-x 1 Admin 197121 0 Oct 26 19:53 .git/
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 a
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 b
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 c
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 d
-rw-r--r-- 1 Admin 197121 0 Oct 26 19:47 def
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 e
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 f
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:35 sample_git
-rw-r--r-- 1 Admin 197121 0 Oct 26 19:04 z

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git add . ; git commit -m "deleted" ; git push origin master
[master e56b430] deleted
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 xyz
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 227 bytes | 227.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:naveenlingampelli456-cmyk/devopsproject-01.git
   851224d..e56b430  master -> master
```

Step 9: Create and Commit Another File

Create a file named mnc:

touch mnc

Stage and commit the file:

git add .

git commit -m "new update"

Verify the directory contents:

ls -l

At this stage, the local repository included the files created during the exercise, with xyz removed.

Screenshot Evidence

```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ touch mnc

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git add .

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git commit -m "new update"
[master 36ecb51] new update
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 mnc

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ ls -l
total 0
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 a
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 b
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 c
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 d
-rw-r--r-- 1 Admin 197121 0 Oct 26 19:47 def
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 e
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:55 f
-rw-r--r-- 1 Admin 197121 0 Oct 26 19:58 mnc
-rw-r--r-- 1 Admin 197121 0 Oct 26 18:35 sample_git
-rw-r--r-- 1 Admin 197121 0 Oct 26 19:04 z

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git push -u origin master
To github.com:naveenlingampelli456-cmyk/devopsproject-01.git
 ! [rejected]        master -> master (fetch first)
error: failed to push some refs to 'github.com:naveenlingampelli456-cmyk/devopsproject-01.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Step 10: Verify SSH Authentication with GitHub

The existing ED25519 SSH key was loaded into the SSH agent:

eval "$(ssh-agent -s)"

ssh-add ~/.ssh/id_ed25519

The screenshot showed the identity being added successfully.

Verify the loaded key:

ssh-add -l -E sha256

The fingerprint shown was:

SHA256:dUN2chC0v/+F8O4dJtv3T92Z34ZB29R3TMgdovZSVs0

The public key was displayed and copied with:

cat ~/.ssh/id_ed25519.pub

clip < ~/.ssh/id_ed25519.pub

The key was registered with the GitHub account. SSH authentication was then tested:

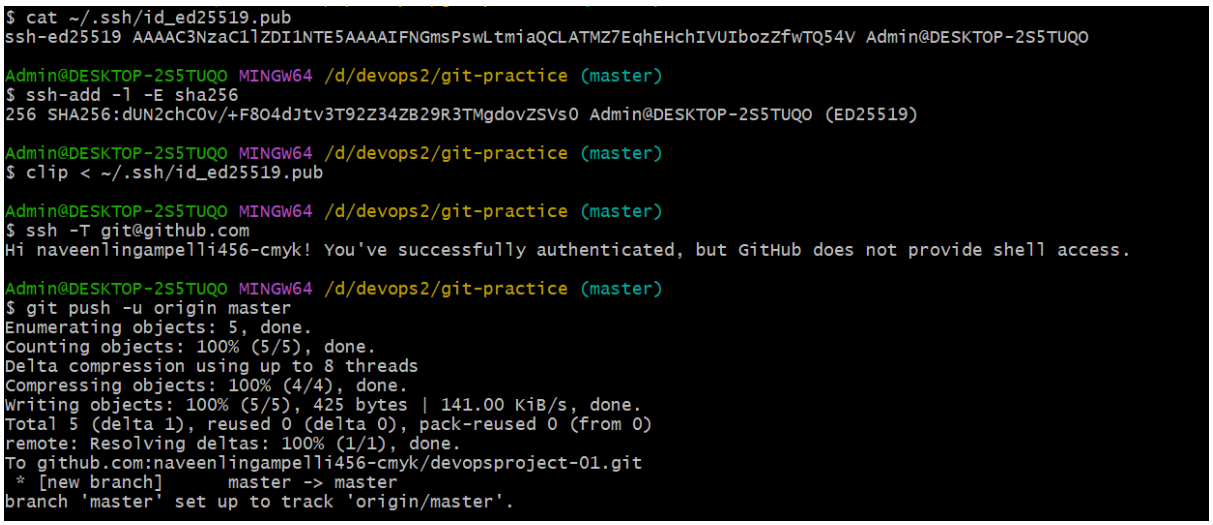
ssh -T git@github.com

The successful output was similar to:

Hi naveenlingampelli456-cmyk! You've successfully authenticated, but GitHub does not provide shell access.

Screenshot Evidence

Screenshot 10



Step 11: Push the Local Repository to GitHub

The remote repository was connected using the SSH URL:

git remote add origin git@github.com:naveenlingampelli456-cmyk/devopsproject-01.git

Verify the remote URL:

git remote -v

Push the local master branch:

git push -u origin master

The screenshot showed successful transfer of objects and creation of the remote master branch.

Step 12: Handle a Rejected Push

A later push was rejected with an error similar to:

! [rejected] master -> master (fetch first)

error: failed to push some refs

This happened because the remote repository contained changes that were not present in the local repository.

The recommended resolution is to fetch and integrate remote changes before pushing again.

The screenshot showed the remote changes being pulled and merged. The resulting output included:

Merge made by the 'ort' strategy.

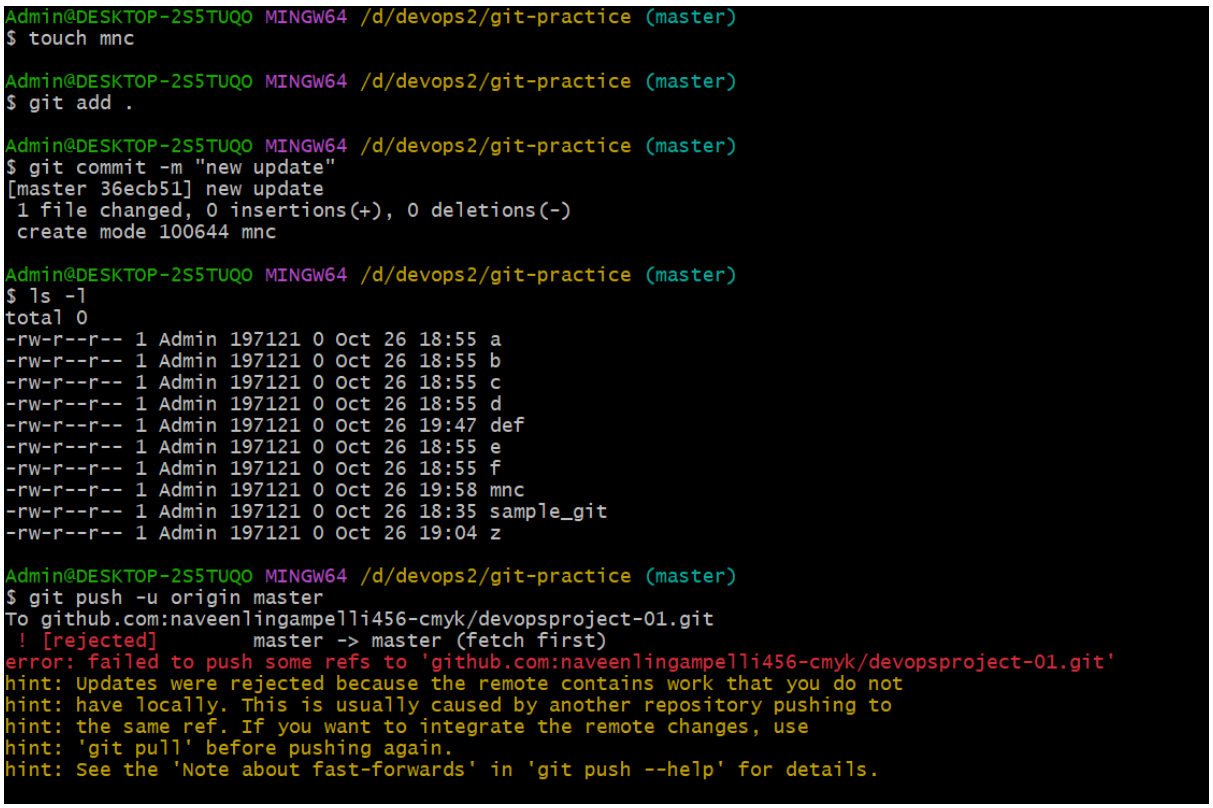
After the merge, the local branch was pushed successfully:

git push -u origin master

The screenshot confirmed that the push completed and the remote branch was updated.

Screenshot Evidence

Screenshot 12:



Step 13: Understand the Staging Area and Reset Command

The screenshot showed additional files named branch and devopsproject-01 being staged.

The repository status displayed the branch as ahead of origin/master by one commit, with staged changes.

The command was executed:

git reset

This unstaged the staged changes while preserving the working-directory files.

The subsequent git status output showed the files as untracked.

Important: git reset without a commit argument normally resets the index to match HEAD; it does not delete the untracked files.

Screenshot Evidence

Screenshot 13:

```
Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   branch
        new file:   devopsproject-01


Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git reset

Admin@DESKTOP-2S5TUQO MINGW64 /d/devops2/git-practice (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        branch
        devopsproject-01/

nothing added to commit but untracked files present (use "git add" to track)
```

Step 14: Verify the Remote Repository

Open the repository:

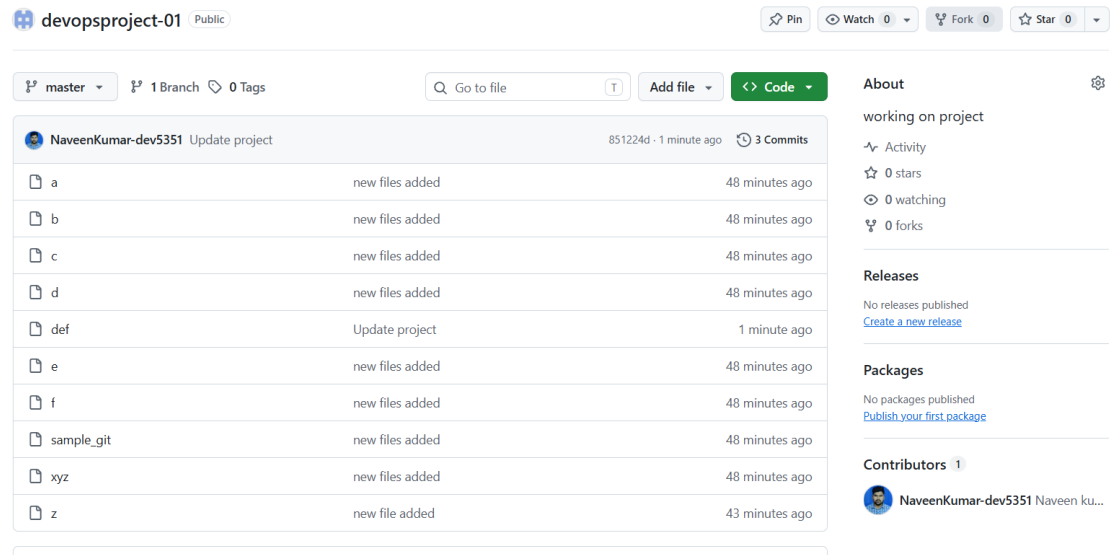
https://github.com/naveenlingampelli456-cmyk/devopsproject-01

The screenshots showed the repository containing files such as a, b, c, d, e, f, sample_git, z, and other files created during the exercise. The repository page also showed commit history and the contributor information displayed by GitHub.

The screenshots confirm that the local changes were pushed to GitHub and that the repository could be viewed through the web interface.

Screenshot Evidence

Screenshot 14:



Validation Steps

1. Verify the Local Repository

Run:

git status

Expected result when no changes are pending:

On branch master

nothing to commit, working tree clean

2. Verify the Commit History

Run:

git log --oneline

This displays the local commit history in a compact format.

3. Verify the Remote Repository

Run:

git remote -v

Expected remote:

origin git@github.com:naveenlingampelli456-cmyk/devopsproject-01.git (fetch)

origin git@github.com:naveenlingampelli456-cmyk/devopsproject-01.git (push)

4. Verify SSH Authentication

Run:

ssh -T git@github.com

Expected result:

GitHub confirms successful SSH authentication and identifies the authenticated account.

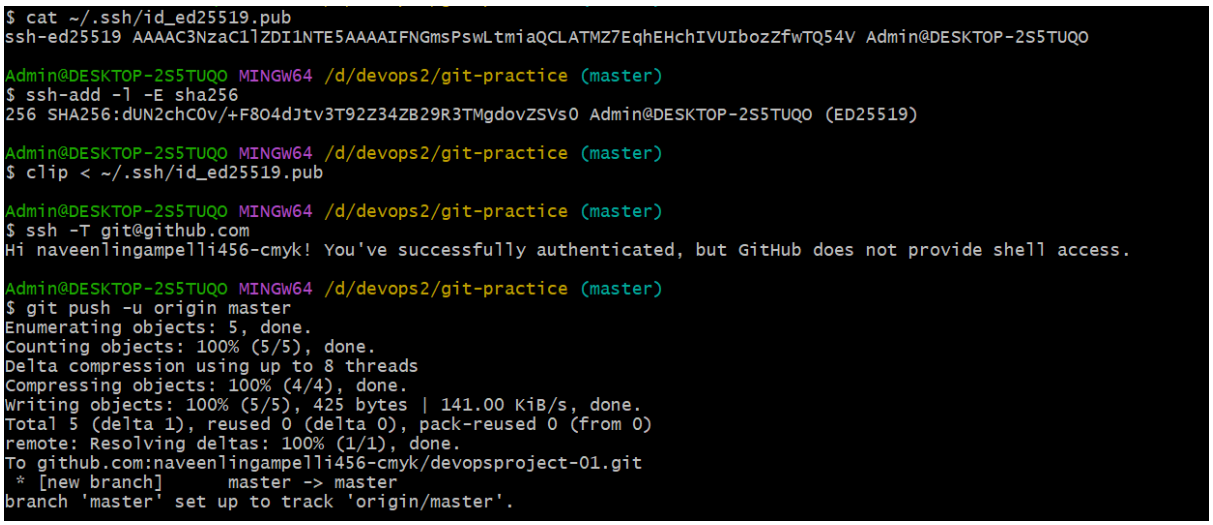
5. Verify the Remote Push

Run:

git push -u origin master

If the local and remote branches are synchronized, Git should report that there is nothing new to push or that the branch is up to date.

Validation Screenshot:



Explanation

Git is a distributed version control system that records changes to files over time. The git init command initializes a repository, git add stages changes, and git commit records a snapshot in the local repository. Commands such as git status and git log help developers inspect pending changes and review the project history. The git reset command can be used to unstage changes without deleting the working files.

GitHub hosts repositories remotely and supports collaboration and code sharing. The git remote command connects the local repository to a remote location, while git push publishes local commits and git pull retrieves and integrates remote changes. SSH authentication provides a secure way to authenticate Git operations without repeatedly entering an account password.

Real-Time Use Case

DevOps engineers use Git to manage application source code, infrastructure-as-code files, automation scripts, and deployment configurations. A typical workflow involves creating a branch, modifying files, staging and committing changes, pushing the branch to GitHub, and collaborating through pull requests. Remote synchronization ensures that local work remains aligned with the shared repository.

Common Errors and Troubleshooting

Error	Cause	Resolution
fatal: not a git repository	The directory has not been initialized or the terminal is in the wrong location	Navigate to the project directory and run git init if needed.
nothing added to commit	No changes have been staged	Run git add for the intended files and retry the commit.

Permission denied (publickey)	SSH key was not offered, was not registered correctly, or was associated with another account	Check the loaded key with ssh-add -l, verify the GitHub SSH key, and test using ssh -T git@github.com.
src refspec master does not match any	The local branch name differs, or no commit exists yet	Run git branch --show-current and create a commit before pushing.
rejected (fetch first)	The remote branch contains commits missing locally	Pull and integrate the remote changes, resolve any conflicts, and push again.
Files remain untracked after git reset	git reset unstages changes but does not remove untracked files	Use git status to inspect the files and explicitly stage or remove them as required.
Wrong contributor shown on GitHub	Commit author information differs from the intended identity	Configure user.name and user.email before creating future commits.
Push rejected due to insufficient access	The authenticated account lacks write permission or the remote URL is incorrect	Verify the repository URL and account access rights.

Conclusion

The Git and GitHub exercise demonstrated repository initialization, file creation, staging, commits, status inspection, commit history review, file deletion, remote configuration, SSH authentication, pushing, pulling, merging, and unstaging changes. The local repository was successfully connected to the public GitHub repository devopsproject-01, and successful push output was captured in the screenshots.

These commands form the foundation of version control workflows used in software development and DevOps.

Final Assignment Summary

Overall Outcome

The following tasks were completed based on the supplied screenshots:

Task	Deliverable	Evidence
Task 1	GitHub account and public repository	GitHub account and repository screenshots
Task 2	Authenticator-app MFA enabled	GitHub 2FA confirmation screenshot
Task 3	Git commands executed and local changes pushed to GitHub	Git Bash outputs and GitHub repository screenshot

Evidence Checklist

- GitHub account created.
- Public repository devopsproject-01 created.
- Authenticator-app MFA enabled.
- Git repository initialized locally.
- Files created and staged.
- Commits created and inspected.
- SSH authentication verified.
- Local changes pushed to GitHub.
- Remote changes pulled and merged after a rejected push.
- GitHub repository contents verified in the browser.

Final Submission

GitHub Repository: https://github.com/naveenlingampelli456-cmyk/devopsproject-01

Submission contents: This task completion document, the relevant terminal screenshots, the GitHub repository screenshot, and the MFA confirmation screenshot.

Security note: Before submitting the document, ensure that screenshots do not expose private SSH keys, recovery codes, MFA setup secrets, or other credentials.